



Department of Artificial Intelligence
22AIE111: OBJECT ORIENTED PROGRAMMING IN JAVA
PROJECT REPORT
CHECKERS GAME

Team Members

Y.Bhasith Sai - CB.SC.U4AIE23264
V.Bhuvanesh - CB.SC.U4AIE23259
G. Harshavardhan - CB.SC.U4AIE23226
U.Sandeep - CB.SC.U4AIE23257

Under the Guidance of

Prof. Dr. Sruthi Guptha

Assistant Professor

Department of Artificial Intelligence

Amrita Vishwa Vidyapeetham, Coimbatore

Coimbatore – 641112

Date of Submission:

Signature of the Project Supervisor:



CERTIFICATE

This certificate at this moment attests that the game-based java project titled "Checkers game" in partial fulfilment of the requirements for the award of Bachelor of Technology in Computer Science & Engineering (Artificial Intelligence) from Amrita Vishwa Vidyapeetham, is an authentic work conducted under my supervision and guidance.

To the best of my knowledge, the content presented in this report is yet to be submitted to any other University or Institute to obtain any degree or diploma.

Date:

Prof. Sruthi Guptha
Dept. of Artificial Intelligence,
Amrita Vishwa Vidyapeetham,
Coimbatore

ACKNOWLEDGEMENT

I extend my heartfelt appreciation to my supervisor, Prof. Dr.Sruthi Guptha, whose exceptional guidance, insightful suggestions, and constructive criticisms have been instrumental throughout this project. Her unwavering support and valuable input significantly enriched the quality of our work.

Special thanks to Dr. Soman K. P. (Dean) and Dr. Subeesh T. (Project Coordinator) for their encouraging words and invaluable suggestions at different project junctures. We are also grateful to the entire team at the Department of Artificial Intelligence, Amrita Vishwa Vidyapeetham, Coimbatore for their support and collaborative spirit.

We are deeply grateful to our colleagues for their continuous flow of innovative ideas and to our dear ones whose encouragement kept us motivated. My sincere thanks go to all those who played a role in helping us complete this project.

AMRITA VISHWA VIDYAPEETHAM
COIMBATORE

Y.Bhasith Sai
V.Bhuvanesh
G. Harshavardhan
U. Sandeep

ABSTRACT

This project aims to develop a Checkers game application using Java programming language. Checkers, also known as Draughts. It is a classic board game played by two players on an 8x8 grid board. The objective of the game is to capture all the opponent's pieces or block them such that they cannot move. The game involves strategic moves, capturing opponent pieces, and planning to defeat the opponent.

The Checkers game application will be implemented using object-oriented principles, with modular design and clear separation of concepts. The game will feature a graphical user interface (GUI) built using Java Swing, providing an intuitive and interactive playing experience. Players will be able to make moves by selecting pieces and choosing valid destination squares.

In this Checkers game implemented in Java different libraries and UI elements are used extensively to create a graphical user interface (GUI) and handle user interaction. The ``javax.swing`` library is used for creating graphical user interface (GUI) components such as buttons, labels, and panels. The ``java.awt`` library is employed for handling basic GUI functionalities like window management, layout management, and drawing graphics. The ``java.awt.event`` library includes classes and interfaces for managing user-generated events such as mouse clicks, key presses, and window events. The ``java.util`` library includes classes for collections, date and time manipulation, random number generation, and other general-purpose utilities.

Contents

S.no	Title	Page no.
1.	Introduction-----	06
2.	Problem Statement-----	07
3.	Objective and Motivation ----- 3.1 Existing system	07-08
4.	Literature review -----	08
5.	System Design with Detailed Explanation ----- 5.1 How to play 5.2 System design	09
6.	System Implementation with Code -----	10 -25
7.	Advantages of the Proposed System -----	25
8.	Results -----	26
8.	Conclusion -----	27
9.	References -----	27

1. Introduction

This project showcases an advanced version of the classic board game, Checkers- built using Java. Using Java's Swing and AWT libraries for creating the graphical user interface (GUI), this project provides an engaging gaming experience. Players have the option to play against each other or challenge the computer. By employing the adaptable GUI components provided by javax.swing, the project enables the creation of user-friendly interfaces, incorporating various elements such as buttons, labels, and panels to enhance usability. Additionally, the java.awt library manages window operations and graphic rendering, ensuring smooth navigation and visual appeal.

An essential aspect of the project is its effective event handling, facilitated through the java.awt.event library. This includes managing user interactions such as mouse clicks and key presses to ensure responsive gameplay and controls. In this project, players can immerse themselves in the strategic gameplay, supported by features such as precise move validation, thorough turn tracking, and comprehensive game state management. These functionalities maintain the integrity of the gameplay, fostering an environment suitable for both competitive and casual play.

Beyond its entertainment value, the Checkers project serves as a testament to Java's capabilities in GUI development, showcasing the language's proficiency in creating intricate interfaces and facilitating seamless interactions.

In summary, the Checkers project seamlessly combines technical innovation with recreational engagement, providing enthusiasts with a platform to enjoy strategic gameplay while highlighting Java's versatility in digital experience creation.

2. Problem Statement

The main aim of this project is to create a Java application for Checkers that faithfully adheres to the game's traditional rules. It will provide a platform where two players can engage in matches on an 8x8 grid, incorporating essential gameplay elements like valid moves and piece promotion.

By employing a turn-based framework, the software ensures fair gameplay, allowing players to alternate moves and enforcing mandatory jumps when necessary. It also accurately determines the game's outcome, signaling victory or defeat accordingly.

The user interface will be designed with intuitiveness in mind, enabling players to interact seamlessly through mouse clicks and receiving informative feedback. Optional features, such as a scoreboard or timer, may be included to enhance the gaming experience.

In essence, the Checkers application aims to deliver an authentic and enjoyable gaming experience by faithfully replicating the traditional game's mechanics while leveraging Java's capabilities to create a smooth and immersive digital platform.

3. Objective and Motivation

3.1 Existing system: (case study on existing systems and identification of their disadvantages)

The existing code implements only two-player checkers game using Java's Swing library for the graphical interface. The game follows some standard checkers rules.

Implementation Details of existing system

- **User Interface:** The game uses Java's Swing framework to create a windowed interface. It includes buttons for starting a new game and resigning, as well as a message display area.
- **Board Representation:** The game board is represented using a 2D array in the CheckersData class. Pieces are stored as integer constants (EMPTY, RED, BLACK, RED_KING, BLACK_KING).
- **Game Logic:** The CheckersData class implements the core game logic, including methods to determine legal moves, execute moves (including capturing opponent pieces), and manage the state of the game board.

Disadvantages:

- It doesn't have option to play in player vs computer game mode.
- Ui is not much interactive and complex.
- Some of the rules are not implemented correctly.
- It doesn't have undo option.

4. Literature review:

1. To create game in Java with a graphical user interface (GUI), begin by designing the game's core logic, such as rules and data structures. We Select a GUI toolkit like Swing or JavaFX and design the visual elements of the game, including the board and pieces. Integrate the game logic with the GUI components, handling user input and updating the display accordingly. Test the game thoroughly to ensure proper functionality and debug any issues. Finally, optimize and refactor the code for readability and performance. With these steps, you can create an engaging and interactive game experience in Java with a GUI.[1]
2. This gives us the rules of checkers in an understandable easy way with pictures, and it also describes about the various strategies like central control, the A – line diagonal, The weak B-line diagonal, etc... And, it contains Opening, Mid-Game, and Endgame which is given by Derek Oldbury.[2]
3. It covers various aspects of the game, including the board setup, piece movement, capturing rules, king promotion, winning conditions, time limits, conduct of players and spectators, as well as guidelines for tournaments and match games. The rules detail specifics such as the size of the board, dimensions of pieces, starting positions, first move determination, handling of time limits, compulsory jumps, king promotions, winning conditions, conduct expectations for players and spectators, tournament formats, and dispute resolution procedures.[3]

5. System Design with Detailed Explanation

5.1 How to play:

- i. Setup: Players sit opposite each other, and each player places their pieces on the 12 dark squares closest to them. The board is set up so that there's a light square in the bottom-right corner for each player.
- ii. Movement: Players take turns moving their pieces diagonally forward along the dark squares. Regular pieces can only move forward, not backward, and they can only move diagonally one square at a time. However, if a piece reaches the opponent's back row, it becomes a "king" and gains the ability to move both forward and backward diagonally.
- iii. Capturing: If a player's piece lands diagonally adjacent to an opponent's piece and the next square beyond it is empty, the player can "capture" the opponent's piece by jumping over it. The captured piece is removed from the board. If there's another opportunity for capturing after a capture, the player must continue capturing with the same piece until no more captures are possible.
- iv. King's Movement: Kings have increased mobility compared to regular pieces. They can move both forward and backward diagonally and can also jump over and capture opponent pieces in any direction diagonally.
- v. Winning the Game: The game ends when one player captures all the opponent's pieces or when the opponent can't make any more moves. If neither of these conditions is met, the game can end in a draw if the same position repeats three times or if both players agree to a draw.

5.2 System design:

- 1) Main class
- 2) Implementation of checkers(constructor)
- 3) Handling Checkers Game Logic and User Interactions
- 4) Implementing Checkers Game Logic in Board Class
- 5) Calculating Legal Moves for Checkers Pieces
- 6) Checking Valid Moves and Captures in Checkers
- 7) Executing Moves in Checkers Game
- 8) Rendering the Checkers Board and Pieces
- 9) Handling Mouse Interactions for Checkers Game
- 10) Determining Legal Moves for Checkers Piece and Highlighting

5. System Implementation with Code

1) Main class

```
import java.awt.*;
import java.awt.event.*;
import javax.swing.*;
import java.util.ArrayList;
import java.util.Arrays;
import java.util.Random;
import java.util.Stack;

public class Checkers extends JPanel {
    private JButton gameModeButton;
    private JButton resignButton;
    private JButton undoButton;
    private JLabel message;

    private Board board;
    private Stack<int[][]> boardStack;

    private boolean playerVsComputer;

    public static void main(String[] args) {
        JFrame window = new JFrame("Checkers");
        Checkers content = new Checkers();
        window.setContentPane(content);
        window.pack();
        Dimension screensize = Toolkit.getDefaultToolkit().getScreenSize();
        window.setLocation((screensize.width - window.getWidth()) / 2,
            (screensize.height - window.getHeight()) / 2);
        window.setDefaultCloseOperation(JFrame.EXIT_ON_CLOSE);
        window.setResizable(false);
        window.setVisible(true);
    }
}
```

The provided Java code defines a class `Checkers` that extends `JPanel` to create a graphical user interface (GUI) for a Checkers game. Within this class, buttons (`gameModeButton`, `resignButton`, `undoButton`) and a label (`message`) are initialized for game control and display purposes. The class also maintains a `Board` object and a stack (`boardStack`) to manage the game state, including moves and potential undo operations. The `main` method sets up a `JFrame` window titled "Checkers" and populates it with an instance of `Checkers` as its content. The window's position is centered on the screen, and the game window is made non-resizable and visible. This setup provides the foundation for developing a Checkers game interface in Java, integrating game logic with GUI components for player interaction.

2) Implementation of checkers(constructor)

```
public Checkers() {
    setLayout(null);
    setPreferredSize(new Dimension(350, 250));
    setBackground(new Color(50, 50, 50)); // Dark background color

    board = new Board();
    add(board);
    gameModeButton = new JButton("Game Mode");
    gameModeButton.addActionListener(this::gameModeClicked);

    resignButton = new JButton("Resign");
    resignButton.addActionListener(this::resignClicked);

    undoButton = new JButton("Undo");
    undoButton.addActionListener(this::undoClicked);

    message = new JLabel("", JLabel.CENTER);
    message.setFont(new Font("Serif", Font.BOLD, 14));
    message.setForeground(Color.WHITE); // White text color
    boardStack = new Stack<>();
    boardStack.push(copyBoardState(board.getBoardState()));
    playerVsComputer = true;
    add(gameModeButton);
    add(resignButton);
    add(undoButton);
    add(message);
    board.setBounds(20, 20, 164, 164);
    gameModeButton.setBounds(210, 20, 120, 30);
    resignButton.setBounds(210, 80, 120, 30);
    undoButton.setBounds(210, 140, 120, 30);
    message.setBounds(0, 200, 350, 30);
}
```

The Checkers() constructor initializes the graphical user interface (GUI) components required for a Checkers game. It sets the layout to null for custom component positioning, specifies the preferred size as 350x250 pixels, and sets a dark background color. Within this constructor, the method initializes a Board object representing the game board and adds it to the panel. It also creates buttons (gameModeButton, resignButton, undoButton) and a label (message) for game control and information display. Action listeners are attached to these buttons to handle user interactions. The constructor sets up a stack (boardStack) to manage board states for potential undo operations and initializes the initial board state. The game mode

is set to `playerVsComputer` as `true` by default. Finally, the components are added to the panel and positioned using absolute coordinates with `setBounds()`. This constructor lays the groundwork for setting up a Checkers game interface in Java, incorporating board display, buttons for game actions, and a message label for status updates.

3) Handling Checkers Game Logic and User Interactions

```
private int[][] copyBoardState(int[][] boardState) {
    int[][] copy = new int[8][8];
    for (int i = 0; i < 8; i++) {
        System.arraycopy(boardState[i], 0, copy[i], 0, 8);
    }
    return copy;
}

private void gameModeClicked(ActionEvent evt) {
    int option = JOptionPane.showOptionDialog(null,
        "Choose game mode:", "Game Mode",
        JOptionPane.YES_NO_OPTION, JOptionPane.QUESTION_MESSAGE,
        null, new String[]{"Player vs. Player", "Player vs.
Computer"}, "Player vs. Player");

    playerVsComputer = (option == JOptionPane.NO_OPTION);

    board.setUpGame();
    if (playerVsComputer) {
        message.setText("Red: Make your move.");
    } else {
        message.setText("Red: Make your move.");
    }
    gameModeButton.setEnabled(false);
    resignButton.setEnabled(true);
    undoButton.setEnabled(true);
    repaint();
    if (!playerVsComputer && !board.isRedsTurn()) {
        computerTurn();
    }
}

private void resignClicked(ActionEvent evt) {
    if (board.isRedsTurn()) {
        gameOver("BLACK wins. RED resigns.");
    } else {
        gameOver("RED wins. BLACK resigns.");
    }
}
```

```

private void undoClicked(ActionEvent evt) {
    System.out.println("Undo button clicked.");
    if (boardStack.size() > 1) {
        boardStack.pop();
        int[][] previousBoardState = boardStack.peek();
        System.out.println("Previous board state: " +
Arrays.deepToString(previousBoardState));
        board.setBoard(previousBoardState);
        board.repaint();
        if (board.isRedsTurn()) {
            message.setText("Red: Make your move.");
        } else {
            message.setText("Black: Make your move.");
        }
    } else {
        System.out.println("No moves to undo.");
    }
}

private void gameOver(String str) {
    message.setText(str);
    if (str.contains("RED")) {
        JOptionPane.showMessageDialog(this, "RED wins!");
    } else {
        JOptionPane.showMessageDialog(this, "BLACK wins!");
    }
    gameModeButton.setEnabled(true);
    resignButton.setEnabled(false);
    undoButton.setEnabled(false);
}

private void computerTurn() {
    if (!board.isRedsTurn()) {
        board.makeComputerMove();
        repaint();
        if (board.isRedsTurn()) {
            message.setText("Red: Make your move.");
        }
    }
    if (isGameOver()) {
        gameOver("RED wins. BLACK cannot move.");
    }
}

private boolean isGameOver() {
    return board.getLegalMoves().isEmpty();
}

```

The provided methods contribute to the functionality of a Checkers game implemented in Java. The `copyBoardState` method creates a deep copy of the current game board state using a 2D array to preserve the state for potential undo operations. The `gameModeClicked` method responds to the "Game Mode" button click, presenting a dialog for the player to choose between "Player vs. Player" or "Player vs. Computer" mode. Based on the selection, it sets the game mode and updates the message label accordingly. The `resignClicked` method handles the resign button click, declaring the game over and displaying the winner based on the current player's turn. The `undoClicked` method allows the player to undo the last move, restoring the previous board state and updating the game message accordingly. The `gameOver` method displays a message indicating the game's end and presents a dialog box to announce the winner. The `computerTurn` method triggers the computer player's move when playing against the computer opponent, and `isGameOver` checks if the game is over by verifying if there are any legal moves left for the current player. Together, these methods manage game flow, player interactions, and game outcome scenarios within the Checkers GUI.

4) Implementing Checkers Game Logic in Board Class

```
private class Board extends JPanel implements ActionListener, MouseListener {
    int[][] getBoardState() {
        return board;
    }
    private static final int EMPTY = 0;
    private static final int RED = 1;
    private static final int RED_KING = 2;
    private static final int BLACK = 3;
    private static final int BLACK_KING = 4;

    private int[][] board;
    private boolean redsTurn;
    private ArrayList<int[]> legalMoves;
    private ArrayList<int[]> highlightedMoves;
    private int selectedRow;
    private int selectedCol;
    void setBoard(int[][] newBoard) {
        for (int i = 0; i < 8; i++) {
            for (int j = 0; j < 8; j++) {
                board[i][j] = newBoard[i][j];
            }
        }
    }
    Board() {
        setBackground(Color.BLACK);
        addMouseListener(this);
    }
}
```

```

        board = new int[8][8];
        setUpGame();
        highlightedMoves = new ArrayList<>();
    }

    void setUpGame() {
        for (int row = 0; row < 8; row++) {
            for (int col = 0; col < 8; col++) {
                if (row % 2 == col % 2) {
                    if (row < 3)
                        board[row][col] = BLACK;
                    else if (row > 4)
                        board[row][col] = RED;
                    else
                        board[row][col] = EMPTY;
                } else {
                    board[row][col] = EMPTY;
                }
            }
        }
        redsTurn = true;
        legalMoves = getLegalMoves();
        selectedRow = -1;
        selectedCol = -1;
    }

    boolean isRedsTurn() {
        return redsTurn;
    }

```

The Board class defined within the Checkers game implementation handles the game logic and graphical representation of the game board. It extends JPanel and implements ActionListener and MouseListener. The class maintains the current state of the board (board) as a 2D array of integers representing empty spaces, red pieces, black pieces, red kings, and black kings. The setBoard method copies a given board state into the current board. The setUpGame method initializes the board with pieces placed according to standard Checkers starting positions, alternating colors on black squares. It also sets up initial game conditions such as legal moves and the player's turn. The isRedsTurn method checks if it's currently red player's turn. Overall, this class encapsulates the essential game mechanics, including setting up the board, managing turns, and determining legal moves for pieces. The implementation integrates user interface events with game state management to create a functional Checkers game environment.

5) Calculating Legal Moves for Checkers Pieces

```
ArrayList<int[]> getLegalMoves() {
    ArrayList<int[]> moves = new ArrayList<>();
    boolean hasCaptureMove = false;
    for (int row = 0; row < 8; row++) {
        for (int col = 0; col < 8; col++) {
            if (board[row][col] == (redsTurn ? RED : BLACK) ||
board[row][col] == (redsTurn ? RED_KING : BLACK_KING)) {
                if (canMove(row, col, row + 1, col + 1)) {
                    moves.add(new int[]{row, col, row + 1, col + 1});
                    if (canCapture(row, col, row + 2, col + 2)) {
                        hasCaptureMove = true;
                    }
                }
                if (canMove(row, col, row - 1, col + 1)) {
                    moves.add(new int[]{row, col, row - 1, col + 1});
                    if (canCapture(row, col, row - 2, col + 2)) {
                        hasCaptureMove = true;
                    }
                }
                if (canMove(row, col, row + 1, col - 1)) {
                    moves.add(new int[]{row, col, row + 1, col - 1});
                    if (canCapture(row, col, row + 2, col - 2)) {
                        hasCaptureMove = true;
                    }
                }
                if (canMove(row, col, row - 1, col - 1)) {
                    moves.add(new int[]{row, col, row - 1, col - 1});
                    if (canCapture(row, col, row - 2, col - 2)) {
                        hasCaptureMove = true;
                    }
                }
            }
        }
    }
    if (hasCaptureMove) {
        ArrayList<int[]> captureMoves = new ArrayList<>();
        for (int[] move : moves) {
            int fromRow = move[0];
            int fromCol = move[1];
            int toRow = move[2];
            int toCol = move[3];
            if (isCaptureMove(fromRow, fromCol, toRow, toCol)) {
                captureMoves.add(move);
            }
        }
        if (!captureMoves.isEmpty()) {
            return captureMoves;
        }
    }
    return moves; }
```


The `getLegalMoves` method within the `Board` class computes the valid moves for Checkers pieces based on the current game state and player's turn. It iterates through the board, identifying pieces owned by the current player (RED for red player's turn and BLACK for black player's turn) or kings (RED_KING and BLACK_KING). For each piece, it checks potential move directions, including regular moves and captures (jump moves). The method determines valid moves by invoking helper methods such as `canMove` and `canCapture`, which assess whether a piece can move to a specified destination square. If a capture move is available for any piece, the method prioritizes capture moves and returns them. Otherwise, it returns regular moves. This method is crucial for enabling gameplay by providing the valid move options available to the current player's pieces based on the Checkers rules.

6) Checking Valid Moves and Captures in Checkers

```
boolean canMove(int fromRow, int fromCol, int toRow, int toCol) {
    if (toRow < 0 || toRow >= 8 || toCol < 0 || toCol >= 8 ||
board[toRow][toCol] != EMPTY)
        return false;
    if (board[fromRow][fromCol] == RED && toRow > fromRow)
        return false;
    if (board[fromRow][fromCol] == BLACK && toRow < fromRow)
        return false;
    return true;}

boolean canCapture(int fromRow, int fromCol, int toRow, int toCol) {
    if (toRow < 0 || toRow >= 8 || toCol < 0 || toCol >= 8 ||
board[toRow][toCol] != EMPTY)
        return false;
    int opponentPiece = (redsTurn ? BLACK : RED);
    int opponentKingPiece = (redsTurn ? BLACK_KING : RED_KING);
    int middlePieceRow = (fromRow + toRow) / 2;
    int middlePieceCol = (fromCol + toCol) / 2;
    if (board[middlePieceRow][middlePieceCol] != opponentPiece &&
board[middlePieceRow][middlePieceCol] != opponentKingPiece)
        return false;
    return true;}

boolean isCaptureMove(int fromRow, int fromCol, int toRow, int toCol)
{
    if (toRow < 0 || toRow >= 8 || toCol < 0 || toCol >= 8)
        return false;
    int opponentPiece = (redsTurn ? BLACK : RED);
    int opponentKingPiece = (redsTurn ? BLACK_KING : RED_KING);
    int middlePieceRow = (fromRow + toRow) / 2;
    int middlePieceCol = (fromCol + toCol) / 2;
```

```

        if (board[middlePieceRow][middlePieceCol] != opponentPiece &&
board[middlePieceRow][middlePieceCol] != opponentKingPiece)
            return false;
        return true;
    }

```

The provided methods `canMove`, `canCapture`, and `isCaptureMove` are integral to the Checkers game logic for verifying whether a piece can make a specific move or capture an opponent's piece. The `canMove` method checks if a move from a specified source position (`fromRow`, `fromCol`) to a destination position (`toRow`, `toCol`) is valid within the bounds of the board and does not collide with another piece. Additionally, it enforces directionality rules based on the type of piece (e.g., red pieces move downward, black pieces move upward). The `canCapture` method validates if a capture move is possible by checking if there's an opponent's piece (or king) to jump over between the source and destination positions. Similarly, the `isCaptureMove` method verifies if a move results in capturing an opponent's piece by checking the presence of an opponent's piece in the middle between the source and destination positions. These methods are essential for implementing the game's rules regarding piece movement and capture interactions, ensuring the validity of player moves during gameplay.

7) Executing Moves in Checkers Game

```

void makeMove(int fromRow, int fromCol, int toRow, int toCol) {
    // Check if it's a capture move
    int middlePieceRow = (fromRow + toRow) / 2;
    int middlePieceCol = (fromCol + toCol) / 2;
    if (Math.abs(fromRow - toRow) == 2 && Math.abs(fromCol - toCol) ==
2) {
        // Capture move, remove the captured piece
        board[middlePieceRow][middlePieceCol] = EMPTY;
    }
    // Move the piece to the new position
    board[toRow][toCol] = board[fromRow][fromCol];
    board[fromRow][fromCol] = EMPTY;
    // Push a copy of the current board state onto the stack
    boardStack.push(copyBoardState(board));

    // Check if the moved piece becomes a king
    if (toRow == 0 && board[toRow][toCol] == RED) {
        board[toRow][toCol] = RED_KING;
    }
    if (toRow == 7 && board[toRow][toCol] == BLACK) {

```

```

        board[toRow][toCol] = BLACK_KING;
    }
}

void makeComputerMove() {
    if (!legalMoves.isEmpty()) {
        ArrayList<int[]> captureMoves = new ArrayList<>();
        for (int[] move : legalMoves) {
            if (isCaptureMove(move[0], move[1], move[2], move[3])) {
                captureMoves.add(move);
            }
        }
        if (!captureMoves.isEmpty()) {
            // Prioritize capturing moves if available
            int[] move = captureMoves.get(0); // Select the first
capturing move
            makeMove(move[0], move[1], move[2], move[3]);
        } else {
            // If no capturing moves available, select a random move
            Random random = new Random();
            int[] move =
legalMoves.get(random.nextInt(legalMoves.size()));
            makeMove(move[0], move[1], move[2], move[3]);
        }
        redsTurn = true; // After the computer's move, it becomes the
player turn
        legalMoves = getLegalMoves(); // Update legal moves for the
player
        repaint(); // Repaint the board
    }
    if (isGameOver()) { // Check if the game is over after the
computer's move
        gameOver("RED wins. BLACK cannot move.");
    }
}
}

```

The `makeMove` and `makeComputerMove` methods in the Checkers game implementation handle the execution of moves, including player and computer moves, within the game board. The `makeMove` method is responsible for moving a piece from a source position (`fromRow`, `fromCol`) to a destination position (`toRow`, `toCol`). If the move is a capture (indicated by the distance of 2 squares), it removes the captured piece by updating the board state. After executing the move, it pushes a copy of the current board state onto the stack for potential undo operations. Additionally, this method checks if a moved piece reaches the opposing end of the board, transforming it into a king. The `makeComputerMove` method allows the computer player to make a move by selecting a capture move if available; otherwise, it

chooses a random move from the legal moves list. After the move, it updates the game state, including toggling the player's turn, updating legal moves, and repainting the board. The method also checks for game over conditions after the computer's move and triggers the game over state if necessary. Together, these methods enable piece movement and game progression in the Checkers implementation, incorporating capture mechanics and computer player decision-making.

8) Rendering the Checkers Board and Pieces

```
public void paintComponent(Graphics g) {
    super.paintComponent(g);
    Graphics2D g2 = (Graphics2D) g;
    g2.setRenderingHint(RenderingHints.KEY_ANTIALIASING,
RenderingHints.VALUE_ANTIALIAS_ON);
    g.setColor(Color.black);
    g.drawRect(0, 0, getSize().width - 1, getSize().height - 1);
    g.drawRect(1, 1, getSize().width - 3, getSize().height - 3);

    for (int row = 0; row < 8; row++) {
        for (int col = 0; col < 8; col++) {
            if (row % 2 == col % 2)
                g.setColor(Color.LIGHT_GRAY);
            else
                g.setColor(Color.GRAY);
            g.fillRect(2 + col * 20, 2 + row * 20, 20, 20);
            switch (board[row][col]) {
                case RED:
                    g.setColor(Color.RED);
                    g.fillOval(4 + col * 20, 4 + row * 20, 15, 15);
                    break;
                case BLACK:
                    g.setColor(Color.BLACK);
                    g.fillOval(4 + col * 20, 4 + row * 20, 15, 15);
                    break;
                case RED_KING:
                    g.setColor(Color.RED);
                    g.fillOval(4 + col * 20, 4 + row * 20, 15, 15);
                    g.setColor(Color.WHITE);
                    g.drawString("K", 7 + col * 20, 16 + row * 20);
                    break;
                case BLACK_KING:
                    g.setColor(Color.BLACK);
                    g.fillOval(4 + col * 20, 4 + row * 20, 15, 15);
                    g.setColor(Color.WHITE);
                    g.drawString("K", 7 + col * 20, 16 + row * 20);
            }
        }
    }
```

```

                                break;
                            }
                        }
                    }

                    if (selectedRow != -1 && selectedCol != -1) {
                        g.setColor(Color.YELLOW);
                        g.drawRect(2 + selectedCol * 20, 2 + selectedRow * 20, 19,
19);
                        g.drawRect(3 + selectedCol * 20, 3 + selectedRow * 20, 17,
17);
                    }

                    if (!highlightedMoves.isEmpty()) {
                        g.setColor(new Color(0, 255, 255, 128)); // Semi-transparent
cyan
                        for (int[] move : highlightedMoves) {
                            int col = move[1];
                            int row = move[0];
                            g.fillRect(2 + col * 20, 2 + row * 20, 20, 20);
                        }
                    }
                }
            }
        }
    }
}

```

The `paintComponent` method defined within the `Board` class is responsible for visually rendering the Checkers game board and pieces on the graphical user interface (GUI). This method overrides the `paintComponent` method inherited from `JPanel` to customize the drawing of the game board. Within this method, the board is drawn as an 8x8 grid of alternating light gray and gray squares to represent the Checkers board pattern. Depending on the state of each square (`board[row][col]`), pieces are rendered as circles (RED and BLACK pieces) or circles with a "K" for kings (RED_KING and BLACK_KING). The positions of the squares and pieces are calculated based on the row and column indices, ensuring proper alignment and spacing within the board. Additionally, if a square is selected (`selectedRow` and `selectedCol` are not -1), it is highlighted with a yellow border. If there are highlighted moves available, these moves are visually indicated by semi-transparent cyan overlays on the corresponding squares. This method encapsulates the visual representation of the Checkers game board and pieces, updating dynamically based on game state changes and user interactions.

9) Handling Mouse Interactions for Checkers Game

```
@Override
public void actionPerformed(ActionEvent evt) {
    // actionPerformed logic if needed
}

@Override
public void mouseClicked(MouseEvent e) {
}

@Override
public void mousePressed(MouseEvent e) {
    int col = (e.getX() - 2) / 20;
    int row = (e.getY() - 2) / 20;
    if (row >= 0 && row < 8 && col >= 0 && col < 8) {
        selectedRow = row;
        selectedCol = col;
        legalMoves = getLegalMovesForPiece(selectedRow, selectedCol);
        highlightAvailableMoves(legalMoves);
        repaint();
    }
}

@Override
public void mouseReleased(MouseEvent e) {
    int col = (e.getX() - 2) / 20;
    int row = (e.getY() - 2) / 20;
    if (selectedRow != -1 && selectedCol != -1 && row >= 0 && row < 8
    && col >= 0 && col < 8) {
        for (int[] move : legalMoves) {
            if (move[2] == row && move[3] == col) {
                makeMove(selectedRow, selectedCol, row, col);
                if (redsTurn && row == 0) {
                    board[row][col] = RED_KING;
                } else if (!redsTurn && row == 7) {
                    board[row][col] = BLACK_KING;
                }
                redsTurn = !redsTurn;
                legalMoves = getLegalMoves();
                selectedRow = -1;
                selectedCol = -1;
                highlightedMoves.clear();
                if (playerVsComputer && !redsTurn) {
                    computerTurn();
                }
                repaint();
            }
        }
    }
}
```

```

                break;
            }
        }
    } else {
        selectedRow = -1;
        selectedCol = -1;
        highlightedMoves.clear();
        repaint();
    }
}

@Override
public void mouseEntered(MouseEvent e) {
}

@Override
public void mouseExited(MouseEvent e) {
}

```

The `mousePressed` and `mouseReleased` methods implemented within the `Board` class manage mouse interactions for the Checkers game GUI. The `mousePressed` method is triggered when a mouse button is pressed within the game board area. It computes the row and column corresponding to the clicked position and updates the `selectedRow` and `selectedCol` variables accordingly. It then retrieves the legal moves for the selected piece using `getLegalMovesForPiece` and highlights these moves on the board using `highlightAvailableMoves` before triggering a repaint of the board.

The `mouseReleased` method is invoked when a mouse button is released after a press. It calculates the row and column of the released position and checks if the release corresponds to a valid move among the legal moves obtained earlier. If a valid move is identified, it executes the move using `makeMove`, updates the board state for potential piece promotion to a king, toggles the player's turn (`redsTurn`), retrieves updated legal moves, clears the selected piece and highlighted moves, and triggers a repaint of the board. Additionally, if playing against the computer (`playerVsComputer` is true) and it's the computer's turn, the method initiates the computer's move using `computerTurn`. If the released position does not correspond to a valid move, the method resets the selected piece and highlights and triggers a repaint.

Together, these mouse event handling methods facilitate player interaction with the Checkers game board, allowing players to select and execute moves through mouse clicks and visually indicating available moves on the board. The methods integrate with game logic and graphical rendering to provide a responsive and interactive user experience for playing Checkers.

10) Determining Legal Moves for Checkers Piece and Highlighting

```
ArrayList<int[]> getLegalMovesForPiece(int row, int col) {
    ArrayList<int[]> moves = new ArrayList<>();
    if (board[row][col] == (redsTurn ? RED : BLACK) || board[row][col]
== (redsTurn ? RED_KING : BLACK_KING)) {
        for (int dRow = -2; dRow <= 2; dRow += 4) {
            for (int dCol = -2; dCol <= 2; dCol += 4) {
                int toRow = row + dRow;
                int toCol = col + dCol;
                int captureRow = row + dRow / 2;
                int captureCol = col + dCol / 2;
                if (canCapture(row, col, toRow, toCol)) {
                    moves.add(new int[]{row, col, toRow, toCol});
                }
            }
        }
        if (moves.isEmpty()) {
            if (canMove(row, col, row + 1, col + 1))
                moves.add(new int[]{row, col, row + 1, col + 1});
            if (canMove(row, col, row - 1, col + 1))
                moves.add(new int[]{row, col, row - 1, col + 1});
            if (canMove(row, col, row + 1, col - 1))
                moves.add(new int[]{row, col, row + 1, col - 1});
            if (canMove(row, col, row - 1, col - 1))
                moves.add(new int[]{row, col, row - 1, col - 1});
        }
    }
    return moves;
}

void highlightAvailableMoves(ArrayList<int[]> moves) {
    highlightedMoves.clear();
    for (int[] move : moves) {
        highlightedMoves.add(new int[]{move[2], move[3]});
    }
}
}
```


The `getLegalMovesForPiece` method within the `Board` class calculates the legal moves available for a selected Checkers piece located at a specified row and col position on the board. It first checks if the piece belongs to the current player (RED or BLACK based on `redsTurn`) or is a king (RED_KING or BLACK_KING). It then examines capturing moves by iterating over potential jump directions (`dRow` and `dCol`), checking if a capture is possible and adding valid capture moves to the list of moves. If no capturing moves are found, the method proceeds to check regular forward and diagonal moves using the `canMove` method and adds them to the list of moves if valid. The method returns the list of legal moves for the selected piece.

The `highlightAvailableMoves` method clears the `highlightedMoves` list and populates it with the destination positions (`toRow` and `toCol`) of the legal moves provided as input. This method is used to visually highlight the available moves on the Checkers board by storing the positions to be highlighted in `highlightedMoves`, which is later utilized by the `paintComponent` method to render the highlighted squares. Together, these methods enable the identification and highlighting of valid moves for a selected Checkers piece, enhancing the user interface to assist players in making informed moves during gameplay.

6. Advantages of the Proposed System

- This implementation supports gameplay functionalities, including player moves, capturing rules, and even a basic computer opponent for single-player mode. Integrates game logic and user interaction to deliver a playable Checkers experience.
- Java Swing provides a robust framework for building desktop applications with rich user interfaces, which enables to deliver a fully functional and interactive Checkers game experience with all correct rules implemented clearly.

7. Results and discussion

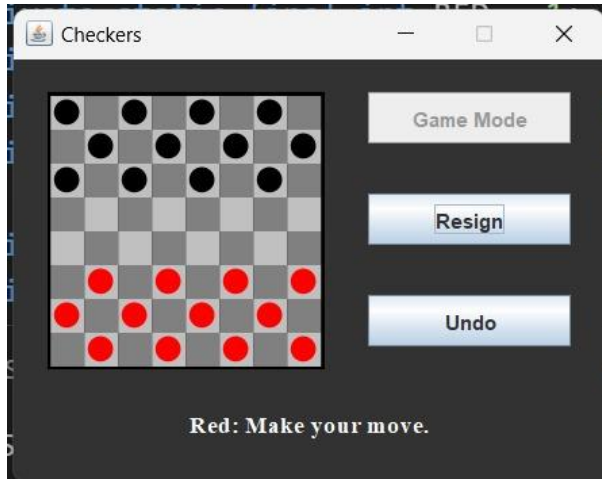


Fig:7.1

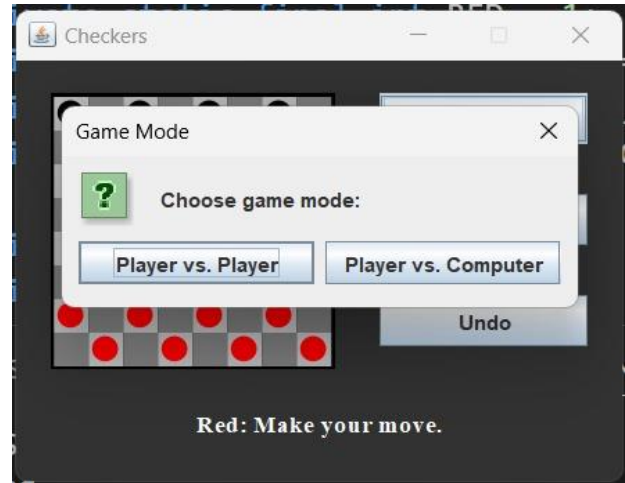


Fig:7.2

The Checkers game implementation in Java has been successfully executed, achieving the intended objectives set at the project's starting. It is now a fully functional application that accurately represents the standard rules of the game. Players can engage in matches on a digital board, with functionalities as in original board. Moreover, the effective user interface (UI) design facilitates seamless player interactions through mouse clicks, while providing informative feedback on legal moves and game status. This aspect enhances the overall user experience, making the game accessible to players of all skill levels.

Firstly by default, game starts in computer vs player game mode and player gets red coins along first move as shown in Fig:7.1. We must drag mouse along click to place where you want to move. And the game follows as normal. We can change game mode after resigning or on game completion.

8. Conclusion

In concluding our discussion on the integration our Java-based checkers game, it becomes evident that this technique serves as a vital component in bridging qualitative game elements using numerical representations for motion of checker pieces. This game code is elegantly structured and thoughtfully implemented. It's divided into clear modules, with the main game logic and GUI elements neatly separated. Meaningful variable names and encapsulation ensure readability and maintainability. The code's event-driven design makes user interaction smooth and responsive, while its error handling features improve reliability. The GUI is visually appealing and intuitive, providing an enjoyable gaming experience. Overall, the code demonstrates a fine balance between functionality and simplicity, making it a beautifully crafted rendition of the classic game.

9. References

1. [Your First Java Game: Step-by-Step Coding Tutorial \(learncodingusa.com\)](https://www.learncodingusa.com/tutorial/your-first-java-game)
[How To Make A Game With Java: Step-By-Step Instructions \(marketsplash.com\)](https://marketsplash.com/tutorial/how-to-make-a-game-with-java)
2. <https://www.usacheckers.com/excerpt-oldbury.php>
3. <http://www.darkfish.com/checkers/rules.html>